

SABERES – Plan completo Hack4Seniors

Oct 1, 2026 · @Valentin

El problema

Las personas de 50+ en Chile **quieren aprender, pero nadie tiene tiempo ni paciencia para enseñarles**; a la vez, **lo que ellas saben no tiene dónde quedar** y muchas se sienten solas y sin un rol. El resultado es dependencia, aislamiento y un conocimiento valioso que se pierde.

Qué pasa	El dato	Fuente
El país envejece rápido	El 14% de la población tiene 65 años o más, y ya hay 79 personas de 65+ por cada 100 menores de 15	Censo 2024 (Forbes Chile)
Quieren aprender	El 82% de las personas de 60+ quiere desarrollar más habilidades digitales, y el 66% se siente obligado a aprender tecnología para no quedar excluido	Radiografía Digital Senior Tech, ClaroVTR y Critería, 2024
Pero no logran hacerlo solas	El 88% tiene internet en casa, pero solo el 42% lo usa y solo el 17% hace trámites digitales de forma independiente; entre los mayores de 80, apenas el 13% usa internet	Infogate, abril 2026
Casi nadie les enseña	Solo el 5% ha recibido capacitación digital formal	El Mostrador, 2025
Se sienten solas	El 49,2% siente soledad no deseada y el 32% no tiene ni un amigo	UC, 2025 · UC
Quieren seguir siendo útiles	El 49% de las personas mayores que trabaja lo hace por bienestar psicológico o para mantenerse activa, no solo por plata	Cipem (ASIMET)

Citas para el pitch:

"Conectar no es incluir. La inclusión comienza cuando alguien puede usar la tecnología para tomar decisiones de manera autónoma." — Paulina Núñez, presidenta del Senado ([La Tercera](#))

"Durante la pandemia se hizo evidente que muchas personas mayores carecían de herramientas para comunicarse o acceder a servicios básicos." — Dra. María Isabel Zavala, Universidad Autónoma de Chile ([Infogate, 2026](#))

Por qué el vínculo entre generaciones importa: la evidencia que recopila la prensa de salud (APA, Harvard, Mayo Clinic) asocia las amistades intergeneracionales con menos depresión en personas mayores y menos estereotipos sobre la vejez en los jóvenes ([Infobae, 2026](#)).

El problema en una frase: *las personas de 50+ quieren aprender y quieren seguir siendo útiles, pero la tecnología las deja solas en lo primero y la sociedad las deja de lado en lo segundo.*

La solución: SABERES

SABERES es una enciclopedia por voz donde las personas de 50+ aprenden paso a paso con una IA paciente, enseñan lo que saben grabando con su propia voz, y los jóvenes aprenden de ellas. Todo se hace hablando, incluso registrarse.

0. Registro por voz (sin teclear nada)

- La persona toca un solo botón: **"Empezar"**. Una voz cálida le pregunta: "¿Cómo se llama?", "¿De qué comuna es?", "¿Qué le gustaría aprender?" y "¿Qué sabe hacer que le gustaría enseñar?".
- La IA convierte esas respuestas en su perfil: nombre, comuna, intereses y saberes. Al final se lo lee para que lo confirme.
- Para entrar después, basta decir su nombre y una **palabra clave que ella elige** (por ejemplo, "copihue"). El número de un familiar es opcional, para avisarle de lo que la persona autorice.

1. APRENDO: guías paso a paso por voz

- La persona pregunta en voz alta: "¿Cómo hago una videollamada?", "¿Cómo sé si un mensaje es una estafa?".
- La IA da **un paso a la vez y espera a que diga "listo"**. Se puede interrumpir: "Repita", "Más lento", "No entendí", y la IA lo explica de otra forma.
- Temas: vida digital, trámites, seguridad y estafas, salud y prevención (ejercicios de equilibrio, cuidar la presión), y las guías que suben otras personas mayores.

2. ENSEÑO: subir lo que sabe, solo hablando

- Toca **"Quiero enseñar"** y lo cuenta como si se lo explicara a un nieto.
- La IA lo convierte en una **guía ordenada**: título, materiales, pasos numerados, consejos y advertencias. Se la lee para que la apruebe o corrija.
- Se publica **con su nombre, foto y comuna**: *"Pan amasado de la señora Rosa, 72 años, Maipú"*.
- **Filtro de seguridad**: en temas delicados (electricidad, gas, salud, remedios) la IA agrega advertencias y deja la guía en revisión antes de publicarla.

3. CONECTO: los jóvenes aprenden de ellas

- Los jóvenes buscan guías (cocina, oficios, huerto, reparaciones, costura, historias de barrio) y aprenden **escuchando la voz de quien sabe**, con el mismo modo paso a paso.
- Pueden hacerle preguntas al autor o pedirle una **clase por videollamada**, coordinada por WhatsApp.
- Cuando alguien aprende, a la persona mayor le llega un WhatsApp: *"Tomás, de 20 años, hizo tu pan amasado y te da las gracias 🙌"*. Y ve su contador: *"34 personas aprendieron contigo este mes"*.

Por qué es preventiva: da **propósito** (un rol y sentirse útil), **mantiene la mente activa** (aprender y explicar), **crea vínculos entre generaciones** y **aumenta la autonomía**: cada guía aprendida es algo que ya no hay que pedirle a nadie.

Frase para el Checkpoint 1 (12:45):

Problema: las personas de 50+ quieren aprender pero nadie tiene paciencia para enseñarles, y al mismo tiempo su conocimiento se pierde y muchas se sienten solas y sin un rol. Usuario: personas de 50 años o más, y jóvenes que quieren aprender de ellas. Solución: SABERES, una enciclopedia por voz donde una IA les enseña paso a paso, convierte lo que ellas saben en guías para jóvenes y les avisa cada vez que alguien aprendió gracias a ellas; todo se usa hablando, incluso el registro.

División del trabajo y cronograma

Tres personas construyen y una se encarga del pitch y del contenido. Si falta tiempo, la prioridad es: **enseñar (relato → guía) → aprender paso a paso → el "gracias" por WhatsApp → registro por voz → vista de jóvenes → filtro de seguridad**.

Persona	Rol	Es dueño de
P1	Interfaz	Vista para personas mayores (botones gigantes), vista para jóvenes, pantalla de guía paso a paso, flujo de registro por voz
P2	IA y backend	Servidor con respaldo Mistral → OpenRouter, endpoints de registro, guía, enseñar y seguridad, prompts
P3	Integraciones	Voz (hablar y escuchar), aviso de "gracias" por Zavu o Telegram, guías precargadas, <code>demo.json</code> , video de respaldo
P4	Pitch y contenido	Datos y citas, validación con mentores, guion, slides, grabar 2 guías reales de personas mayores si es posible, README

Hora	P1 Interfaz	P2 IA y backend	P3 Integraciones	P4 Pitch y contenido
10:30–11:30	Vista personas mayores e inicio	Repo, servidor y <code>llm()</code> con respaldo	Llaves; probar voz, Zavu y Telegram	Problema, usuaria y datos
11:30–12:45	Pantalla de guía paso a paso	<code>/api/ensenar</code> (relato → guía)	<code>hablar()</code> y <code>escuchar()</code> ; 5 guías precargadas	Mentorías: validar y anotar 1 cita
12:45				Checkpoint enviado
13:00–14:00	Flujo "Quiero enseñar"	<code>/api/guia</code> (paso a paso y dudas)	Aviso de "gracias" por Zavu	Buscar a 1 o 2 personas mayores que graben una guía
14:00–15:30	Registro por voz y vista de jóvenes	<code>/api/registro</code> y filtro de seguridad	<code>demo.json</code> y pruebas de punta a punta	Taller de pitch (PI03) ; después slides
15:30	Se congelan las funciones	Arreglar errores	Grabar video de respaldo	Slides finales

Hora	P1 Interfaz	P2 IA y backend	P3 Integraciones	P4 Pitch y contenido
15:45–16:30	Ensayo	README y repo público	Probar en el escenario	Ensayar 3 veces con cronómetro
16:45		Repo público y slides subidas		
17:15	Maneja la vista de mayores	Maneja la vista de jóvenes	Tiene el celular que recibe el "gracias"	Presenta

P2 – IA y backend

Un servidor Node + Express guarda las llaves, habla con Mistral (y con OpenRouter si Mistral falla) y guarda usuarios y guías en archivos JSON. Meta a las 12:45: `/api/ensinar` convierte un relato hablado en una guía ordenada.

Crear el proyecto (primeros 15 minutos):

```
mkdir saberes && cd saberes
npm init -y && npm pkg set type=module
npm install express dotenv
mkdir -p public data
echo '[]' > data/usuarios.json && echo '[]' > data/guias.json
printf ".env\nnode_modules\n" > .gitignore
```

.env (nunca se sube):

```
MISTRAL_API_KEY=...
OPENROUTER_API_KEY=...
OPENROUTER_MODEL=google/gemini-2.0-flash-001 # o el que aparezca en
openrouter.ai/models
ZAVUDEV_API_KEY=...
ZAVU_CANAL=whatsapp # o sms
NUMERO_DEMO=+569XXXXXXXXX # celular que recibe el "gracias" en la demo
TELEGRAM_TOKEN=...
TELEGRAM_CHAT_ID=...
AVISO=zavu # o telegram
```

1. Base, datos y respaldo automático (`server.js`):

```

import express from "express";
import fs from "fs";
import "dotenv/config";

const app = express();
app.use(express.json({ limit: "5mb" }));
app.use(express.static("public"));

const leer = (f) => JSON.parse(fs.readFileSync(`data/${f}.json`, "utf8"));
const guardar = (f, d) => fs.writeFileSync(`data/${f}.json`,
JSON.stringify(d, null, 2));
const id = () => Math.random().toString(36).slice(2, 9);
const normal = (s = "") => s.toLowerCase().normalize("NFD").replace(/[-_]/g,
 "").trim();

async function llm({ system, user, json = true }) {
  const proveedores = [
    { url: "https://api.mistral.ai/v1/chat/completions", key:
process.env.MISTRAL_API_KEY, model: "mistral-small-latest" },
    { url: "https://openrouter.ai/api/v1/chat/completions", key:
process.env.OPENROUTER_API_KEY, model: process.env.OPENROUTER_MODEL },
  ];
  for (const p of proveedores) {
    try {
      const r = await fetch(p.url, {
        method: "POST",
        headers: { "Content-Type": "application/json", Authorization: `Bearer
${p.key}` },
        body: JSON.stringify({
          model: p.model, temperature: 0.3,
          ...(json && { response_format: { type: "json_object" } }),
          messages: [{ role: "system", content: system }, { role: "user",
content: user }],
        }),
        signal: AbortSignal.timeout(10000),
      });
      if (!r.ok) throw new Error(`${p.url} respondió ${r.status}`);
      const txt = (await r.json()).choices[0].message.content;
      return json ? JSON.parse(txt.slice(txt.indexOf("{"),
txt.lastIndexOf("}") + 1)) : txt;
    } catch (e) { console.warn("Proveedor falló:", e.message); }
  }
  throw new Error("Sin proveedor disponible");
}

```

2. Registro e ingreso por voz:

```

app.post("/api/registro", async (req, res) => {
  // req.body.respuestas = { nombre: "...", comuna: "...", aprender: "...",
  enseñar: "...", clave: "...", telefono: "..."}
  const p = await llm({
    system: `Ordena las respuestas habladas de una persona que se registra en
  SABERES. Responde SOLO JSON:
  {"nombre":"nombre propio con mayúscula","comuna":"...","intereses":["temas
  que quiere aprender"],
  "saberes":["temas que sabe y puede enseñar"],"palabra_clave":"una sola
  palabra en minúsculas",
  "telefono":"+569... o null","bienvenida":"2 frases cálidas, tratando de
  usted, que resuman su perfil"}`,
    user: JSON.stringify(req.body.respuestas),
  });
  const usuarios = leer("usuarios");
  const u = { id: id(), ...p, enseñados: 0 };
  usuarios.push(u); guardar("usuarios", usuarios);
  res.json(u);
});

app.post("/api/entrar", (req, res) => {
  const u = leer("usuarios").find((x) =>
    normal(x.nombre).includes(normal(req.body.nombre)) &&
    normal(x.palabra_clave) === normal(req.body.clave));
  res.json(u || { error: "No lo encontré. ¿Me repite su nombre y su palabra
  clave?" });
});

```

3. Enseñar: el relato hablado se convierte en guía (la pieza estrella):

```

app.post("/api/ensenar", async (req, res) => {
  const { usuarioId, relato } = req.body;
  const autor = leer("usuarios").find((u) => u.id === usuarioId) || { nombre:
  "Anónimo", comuna: "" };
  const g = await llm({
    system: `Eres el editor de SABERES. Una persona mayor explicó algo
    hablando. Conviértelo en una guía clara,
    SIN inventar pasos que no dijo y conservando su forma de decir las cosas.
    Responde SOLO JSON:
    {"titulo":"...", "categoria":"cocina|oficios|huerto|hogar|digital|salud|histor
    ias|otro",
    "materiales":["..."], "pasos":["un paso por frase, en orden"], "consejos":
    ["trucos que mencionó"],
    "advertencias":["cuidados de seguridad"], "riesgo":"bajo|alto",
    "resumen_voz":"2 frases para leerle la guía y pedirle que la confirme"}
    riesgo es "alto" si trata de electricidad, gas, remedios, diagnósticos o algo
    que pueda causar daño.` ,
    user: relato,
  });
  const guias = leer("guias");
  const guia = { id: id(), ...g, autor: autor.nombre, comuna: autor.comuna,
  autorId: usuarioId,
    estado: g.riesgo === "alto" ? "en revisión" : "publicada",
  aprendieron: 0 };
  guias.push(guia); guardar("guias", guias);
  res.json(guia);
});

```

4. Buscar la guía que la persona pide:

```

app.post("/api/buscar", async (req, res) => {
  const guias = leer("guias").filter((g) => g.estado === "publicada");
  const r = await llm({
    system: `Elige la guía que mejor responde la pregunta. Responde SOLO
    JSON: {"id":"id de la guía o null"}`,
    user: `Pregunta: "${req.body.pregunta}". Guías:
    ${JSON.stringify(guias.map((g) => ({ id: g.id, titulo: g.titulo })))}`,
  });
  res.json(guias.find((g) => g.id === r.id) || { error: "No encontré esa guía
  todavía. ¿Quiere que se la pida a la comunidad?" });
});

```

5. Ayuda durante la guía ("no entendí", "¿cómo era eso?"):

```
app.post("/api/ayuda", async (req, res) => {
  const { guia, paso, duda } = req.body;
  const texto = await llm({
    json: false,
    system: `Eres un profesor paciente de SABERES. Explica de otra forma el
paso actual, en máximo 2 frases
simples, tratando de usted. No adelantes pasos.`,
    user: `Guía: ${guia.titulo}. Paso ${paso + 1}: ${guia.pasos[paso]}. La
persona dijo: "${duda}"`,
  });
  res.json({ texto });
});
```

6. "Aprendí": suma al contador y avisa al autor (la función `avisar()` está en la sección de P3):

```
app.post("/api/aprendi", async (req, res) => {
  const guias = leer("guias");
  const g = guias.find((x) => x.id === req.body.guiaId);
  g.aprendieron++; guardar("guias", guias);
  await avisar(`🙏 ${req.body.aprendiz} aprendió "${g.titulo}" gracias a
usted, ${g.autor}. Ya son ${g.aprendieron} personas que aprendieron con su
guía.`);
  res.json({ ok: true, aprendieron: g.aprendieron });
});

app.get("/api/guias", (req, res) => res.json(leer("guias").filter((g) =>
g.estado === "publicada")));
app.listen(3000, () => console.log("SABERES en http://localhost:3000"));
```

P1 - Interfaz

Dos vistas en la misma app web: una **para personas mayores** (todo con voz y botones gigantes) y otra **para jóvenes** (buscador de guías con la foto y la historia de cada autor). Usan las funciones `hablar()` y `escuchar()` de P3.

Pantallas de la vista para personas mayores:

1. **Bienvenida:** un solo botón gigante **"Empezar"** (y abajo, más chico, "Ya tengo cuenta").
2. **Registro por voz:** una pregunta a la vez en letra grande, con subtítulo de lo que la persona va diciendo y botones **"Sí, está bien"** / **"Repetir"**.
3. **Inicio:** tres botones gigantes: 🎓 **Aprender**, 🗣️ **Quiero enseñar** y ★ **Mis guías** (con el contador "34 personas aprendieron contigo").

4. **Guía paso a paso:** el paso actual en letra enorme, el número ("Paso 3 de 7") y 4 botones: **Listo, Repetir, Más lento y No entendí.**
5. **Enseñar:** un botón rojo grande de grabar, el texto apareciendo mientras habla, y al final la guía ordenada con **"Publicar" / "Corregir"**.

Vista para jóvenes: tarjetas con la foto del autor, su nombre, comuna y edad, el título de la guía y cuántos aprendieron; al abrirla, el mismo modo paso a paso y el botón **"¡Aprende! Dar las gracias"**.

Estilos clave (`estilos.css`):

```
.mayor { font-size: 28px; }
.mayor button { min-height: 84px; font-size: 30px; border-radius: 20px; font-weight: 700; }
.paso-actual { font-size: 40px; line-height: 1.3; }
.grabando { animation: latido 1s infinite; background: #c62828; color: #fff; }
@keyframes latido { 50% { transform: scale(1.05); } }
```

Registro por voz (`app.js`):

```
const PREGUNTAS = [
  ["nombre", "Bienvenido a SABERES. ¿Cómo se llama?"],
  ["comuna", "¿De qué comuna es?"],
  ["aprender", "¿Qué le gustaría aprender?"],
  ["enseñar", "¿Qué sabe hacer que le gustaría enseñar a otros?"],
  ["clave", "Elija una palabra que le sea fácil de recordar, por ejemplo el nombre de una flor. Esa será su clave."],
  ["telefono", "Si quiere, dígame el celular de un familiar. Si no, diga 'no'."],
];

async function registrarPorVoz() {
  const respuestas = {};
  for (const [campo, pregunta] of PREGUNTAS) {
    mostrarPregunta(pregunta);
    await hablar(pregunta);
    respuestas[campo] = await escucharConRespaldo(); // si falla el micrófono, muestra un campo para escribir
  }
  const u = await post("/api/registro", { respuestas });
  await hablar(u.bienvenida);
  usuario = u;
  irA("inicio");
}
```

Guía paso a paso con comandos de voz:

```

async function seguirGuia(guia) {
  let paso = 0;
  await hablar(`Vamos a aprender: ${guia.titulo}. Enseña ${guia.autor}.
  Necesita: ${guia.materiales.join(", ") || "nada especial"}`);
  while (paso < guia.pasos.length) {
    mostrarPaso(guia, paso);
    await hablar(`Paso ${paso + 1}. ${guia.pasos[paso]}`);
    const orden = normal(await esperarOrden()); // voz o botones:
    listo / repetir / más lento / no entendí
    if (orden.includes("listo") || orden.includes("siguiente")) paso++;
    else if (orden.includes("lento")) { velocidadVoz = 0.75; }
    else if (orden.includes("repit")) continue;
    else {
      const { texto } = await post("/api/ayuda", { guia, paso, duda: orden
    });
    await hablar(texto);
  }
  if (guia.consejos?.length) await hablar(`Un consejo de ${guia.autor}:
  ${guia.consejos[0]}`);
  await hablar("¡Lo logró! ¿Quiere darle las gracias a quien le enseñó?");
  if (await confirmar()) await post("/api/aprendi", { guiaId: guia.id,
  aprendiz: usuario?.nombre || "Alguien" });
}

```

Enseñar:

```

async function enseñar() {
  await hablar("Cuénteme lo que quiere enseñar, como si se lo explicara a un
  nieto. Cuando termine, toque el botón rojo.");
  const relato = await grabarHastaQueToque(); // reconocimiento
  de voz continuo (ver P3)
  const guia = await post("/api/ensenar", { usuarioId: usuario.id, relato });
  mostrarGuia(guia);
  await hablar(`${guia.resumen_voz} ¿La publico así?`);
  if (await confirmar()) await hablar(guia.estado === "publicada"
    ? "¡Listo! Su guía ya está publicada con su nombre."
    : "Su guía quedó en revisión porque trata un tema delicado. Le avisamos
    cuando se publique.");
}

```

Accesibilidad que se menciona en el pitch: letra de 28 px o más, botones de 84 px, alto contraste, voz y texto siempre juntos, un paso a la vez y nunca un límite de tiempo.

P3 - Integraciones

P3 hace que todo se pueda usar hablando, que el "gracias" llegue de verdad a un celular y que la demo nunca se caiga.

1. Voz para toda la app (`public/voz.js` , Web Speech API de Chrome, sin costo):

```
let velocidadVoz = 0.9;
const normal = (s = "") => s.toLowerCase().normalize("NFD").replace(/[-&#x2013;]/g, "").trim();
const post = async (url, body) => (await fetch(url, { method: "POST",
  headers: { "Content-Type": "application/json" }, body: JSON.stringify(body)
})).json();

function hablar(texto) {
  return new Promise((ok) => {
    speechSynthesis.cancel();
    const u = new SpeechSynthesisUtterance(texto);
    u.lang = "es-CL"; u.rate = velocidadVoz;
    u.voice = speechSynthesis.getVoices().find((v) =>
v.lang.startsWith("es")) || null;
    u.onend = ok; speechSynthesis.speak(u); mostrarSubtitulo(texto);
  });
}

function escuchar() {
  return new Promise((ok, err) => {
    const R = window.SpeechRecognition || window.webkitSpeechRecognition;
    const r = new R(); r.lang = "es-CL";
    r.onresult = (e) => ok(e.results[0][0].transcript); r.onerror = err;
    r.start();
  });
}

// Si el micrófono falla por ruido, aparece un campo grande para escribir
const escucharConRespaldo = () => escuchar().catch(() => pedirPorTeclado());

// Órdenes durante la guía: gana lo primero que llegue, la voz o un botón
const esperarOrden = () => Promise.race([escucharConRespaldo(),
esperarBotonOrden()]);

const confirmar = () => Promise.race([
  escucharConRespaldo().then((r) => /s[íi]|bueno|ya|dale|claro/i.test(r)),
  esperarBotonSiNo(),
]);

// Grabación larga para "Quiero enseñar": junta todo hasta que toquen el
```

```

botón rojo
function grabarHastaQueToque() {
  return new Promise((ok) => {
    const R = window.SpeechRecognition || window.webkitSpeechRecognition;
    const r = new R(); r.lang = "es-CL"; r.continuous = true;
    r.interimResults = true;
    let texto = "";
    r.onresult = (e) => {
      texto = [...e.results].map((x) => x[0].transcript).join(" ");
      mostrarTranscripcion(texto);
    };
    r.onend = () => { if (!terminado) r.start(); }; // Chrome corta tras
silencios: se reanuda solo
    let terminado = false;
    document.querySelector("#btn-terminar").onclick = () => { terminado =
true; r.stop(); ok(texto); };
    r.start();
  });
}

```

2. El "gracias" real por WhatsApp (en `server.js`):

```

async function avisar(texto) {
  try {
    if (process.env.AVISO !== "zavu") throw new Error("usar Telegram");
    const r = await fetch("https://api.zavu.dev/v1/messages", {
      method: "POST",
      headers: { Authorization: `Bearer ${process.env.ZAVUDEV_API_KEY}`,
"Content-Type": "application/json" },
      body: JSON.stringify({ to: process.env.NUMERO_DEMO, text: texto,
channel: process.env.ZAVU_CANAL }),
    });
    if (!r.ok) throw new Error("Zavu respondió " + r.status);
  } catch (e) {
    console.warn("Usando Telegram:", e.message);
    await
fetch(`https://api.telegram.org/bot${process.env.TELEGRAM_TOKEN}/sendMessage`
, {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ chat_id: process.env.TELEGRAM_CHAT_ID, text:
texto }),
    }).catch(() => {});
  }
}

```

Sobre WhatsApp con Zavu: requiere conectar una cuenta de WhatsApp Business. Si lo logran, el celular que recibe el "gracias" debe **escribir primero** al número del negocio para abrir la ventana de 24 horas. Si no alcanzan, usen `ZAVU_CANAL=sms` o `AVISO=telegram`. Para el Telegram: hablar con @BotFather, `/newbot`, escribirle al bot desde el celular y sacar el `chat.id` en `https://api.telegram.org/bot<TOKEN>/getUpdates`.

3. Guías precargadas (`data/guias.json`) para que la demo se vea viva desde el inicio. Lleguen con estas 5, con autores ficticios:

Guía	Categoría	Autor (ficticio)
Cómo hacer una videollamada por WhatsApp	digital	Equipo SABERES
Cómo saber si un mensaje es una estafa	digital	Equipo SABERES
Pan amasado	cocina	Señora Rosa, 72, Maipú
Plantar tomates en macetero	huerto	Don Luis, 68, La Florida
Coser un botón que se salió	hogar	Señora Carmen, 75, Ñuñoa

Cada una con el formato de `/api/enseñar`: `titulo`, `categoria`, `materiales`, `pasos`, `consejos`, `advertencias`, `riesgo`, `autor`, `comuna`, `estado`: "publicada" y `aprendieron` (por ejemplo, 34).

4. Respaldo para que la demo nunca se caiga (`public/demo.json`): la respuesta perfecta de `/api/registro` para la persona de la demo, de `/api/enseñar` para el relato que se va a contar en vivo, y de `/api/ayuda` para una duda típica. Si la IA tarda más de 10 segundos, el frontend usa estas respuestas.

5. Video de respaldo: grabar la demo completa a las 15:30, por si falla todo.

P4 – Pitch, demo y preguntas del jurado

El pitch gira en torno a una demo en vivo de 90 segundos en la que una persona mayor enseña, un joven aprende y un celular real recibe el "gracias".

Validación en las mentorías (11:30): preguntar a los especialistas *¿qué es lo que más les cuesta aprender a las personas mayores?, ¿qué les da sentido después de jubilar? y ¿conocen a alguien que quiera enseñar algo?*. Anotar una frase textual para el pitch. Si alguna persona mayor del evento acepta grabar una guía real, es oro para la demo.

Montaje: un notebook proyecta la vista para personas mayores y otro la vista para jóvenes. Antes de empezar, P3 le entrega a un jurado el celular que recibirá el "gracias".

La demo (4 momentos):

1. **Registro por voz:** la "señora Rosa" (un integrante o un jurado) toca "Empezar" y responde 4 preguntas hablando. La app le lee su perfil.
2. **Enseñar:** Rosa toca "Quiero enseñar" y explica en 40 segundos cómo hace algo (por ejemplo, sopaipillas). **En pantalla aparece la guía ordenada**, con pasos, consejos y su nombre.
3. **Aprender:** en la vista de jóvenes, un jurado abre esa guía y la IA lo lleva paso a paso; dice "no entendí" y la IA lo explica de otra forma.
4. **El gracias:** el jurado toca "¡Aprendí!" y **suenan el celular** con el mensaje: *"Tomás aprendió sus sopaipillas gracias a usted, señora Rosa"*.

Tiempo	Contenido
0:00–0:25	Gancho: "¿Quién aprendió algo de su abuela que no está en ningún tutorial?" Ese conocimiento se está perdiendo, y al mismo tiempo el 82% de las personas de 60+ quiere aprender más y casi nadie les enseña.
0:25–0:55	Problema: solo el 17% hace trámites digitales sola, el 49% siente soledad y muchas pierden su rol al jubilar. Cita del Senado: "Conectar no es incluir".
0:55–2:25	Demo en vivo (4 momentos).
2:25–3:05	Cómo funciona: IA (Mistral, con respaldo en OpenRouter) que guía con paciencia y convierte relatos en guías, filtro de seguridad, avisos reales con Zavu; todo por voz, incluso el registro.
3:05–3:40	Impacto y escala: municipios y clubes de adulto mayor, universidades (horas de vinculación con el medio para jóvenes), cajas de compensación. Cada guía es un saber que queda guardado.
3:40–4:00	Cierre: "Las personas mayores no son solo alumnas de la tecnología: son la enciclopedia más grande que tenemos. SABERES les enseña con paciencia y les da una voz para enseñar."

Preguntas probables del jurado:

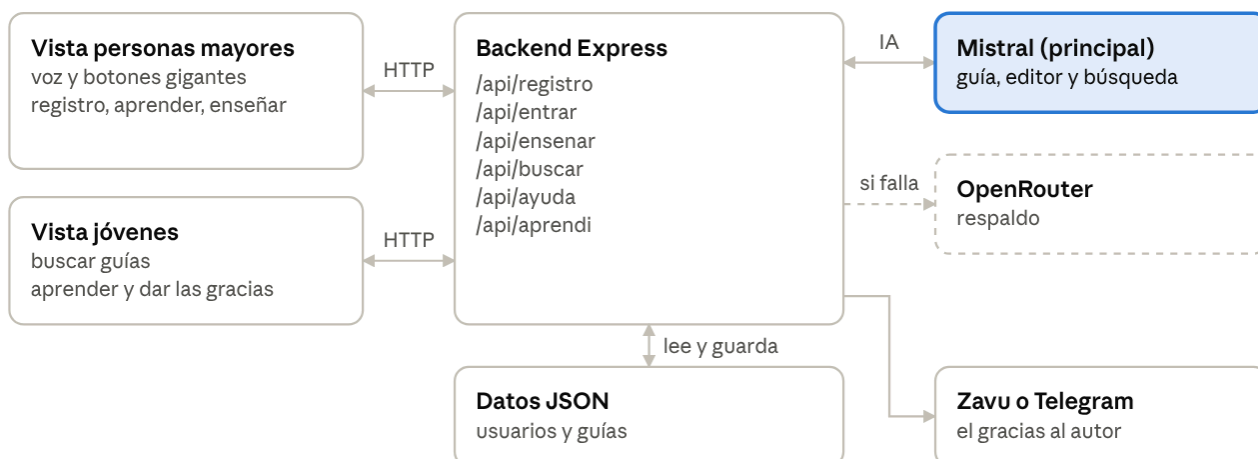
- **¿En qué se diferencia de YouTube?** → En YouTube hay que buscar, mirar y pausar. Aquí la IA guía paso a paso, espera, responde dudas, y quien enseña puede hacerlo solo hablando, sin grabar videos ni editar.

- **¿Y si alguien enseña algo peligroso o falso?** → La IA no inventa pasos, agrega advertencias y deja en revisión los temas delicados (electricidad, gas, salud, remedios).
- **¿Cómo es seguro registrarse por voz?** → Para el prototipo usamos nombre + palabra clave; la versión real sumaría verificación con el celular o un familiar.
- **¿Cómo llegan las personas mayores?** → Por municipios, clubes de adulto mayor y CESFAM, y por sus propios nietos, que son quienes más aprenden de ellas.
- **¿Quién paga?** → Municipios y cajas de compensación como programa de envejecimiento activo; universidades por la vinculación con el medio de sus estudiantes.

Arquitectura

Las dos vistas hablan con un solo backend; el backend usa Mistral para guiar, ordenar y buscar guías, cambia a OpenRouter si falla y avisa al autor por Zavu o Telegram.

La IA guía, ordena las guías y avisa al autor; todo pasa por voz



arquitectura de SABERES · 7 piezas

Si la red falla, el frontend usa `demo.json`. Los datos se guardan en dos archivos JSON (`usuarios.json` y `guias.json`), suficiente para el prototipo.

Riesgos, plan B y lista de entrega

Un flujo completo funcionando perfecto (enseñar → aprender → gracias) vale más que todas las funciones a medias.

Riesgo	Plan B	Responsable
Mistral tarda o falla	El backend cambia solo a OpenRouter; si también falla, <code>demo.json</code>	P2
Se cae internet del evento	Celular con datos compartiendo internet + <code>demo.json</code>	P3
El micrófono no entiende por el ruido	Campo grande para escribir y botones Listo / Repetir / Sí / No	P1
Chrome corta la grabación larga	<code>grabarHastaQueToque()</code> se reanuda sola; el relato de la demo dura menos de 1 minuto	P3
WhatsApp de Zavu no queda conectado	<code>ZAVU_CANAL=sms</code> o <code>AVISO=telegram</code>	P3
Chrome bloquea el audio	Hacer un clic en la página antes de empezar	P1
Todo falla	Video de respaldo grabado a las 15:30	P3

Lista de entrega:

- ☐ Checkpoint 1 enviado a las 12:45
- ☐ Repo en GitHub **público** (github.com/restry12/Caravean), sin el `.env`
- ☐ README actualizado a SABERES, con capturas y cómo correrlo (`npm install` y `node server.js`)
- ☐ 5 guías precargadas y `demo.json` listos
- ☐ Slides subidas antes de las 16:45, con los datos del problema, capturas y el diagrama de arquitectura
- ☐ Video de respaldo de la demo
- ☐ Pitch ensayado 3 veces en 4:00 exactos
- ☐ Celular que recibe el "gracias" cargado y con el chat abierto